Security Audit Report

AtomOne v4 Changes

v1.0

April 23, 2026

Table of Contents

Table of Contents 2
License 4
Disclaimer 5
Introduction 6
Purpose of This Report 6
Codebase Submitted for the Audit 6
Methodology 8
Functionality Overview 8
How to Read This Report 9
Code Quality Criteria 10
Summary of Findings 11
Detailed Findings 15
1. Duplicate header resubmission falsely triggers misbehavior detection and freezes the Gno light client 15
2. Equal-height equivocation detection is completely disabled due to using the parent block ID instead of the commit block ID 15
3. VerifyLightCommit incorrectly validates signatures by index, breaking non-adjacent header verification 16
4. Proto3 CommitSig encoding cannot represent absent Gno.land precommits, preventing conversion of blocks 17
5. MaxGovernors parameter is defined but never checked 18
6. Inverted error check in staking hooks disables governor auto-deactivation 18
7. Nested authz MsgExec bypasses GovVoteDecorator minimum stake check 19
8. Minimum stake requirement for governance voting is bypassed via cosmos.gov.v1 message types 20
9. Genesis import doubles active governor validator shares 20
10. Missing event emission in UpdateParams reduces off-chain observability 21
11. MsgUpdateParams does not validate params, allowing chain-halting parameter configuration 21
12. MinLearningRate Nil check uses the wrong field in parameter validation 22
13. Panic recovery in UpdateBaseGasPrice can leave an inconsistent state 22
14. No upper bound check on minted photons relative to MaxSupply 23
15. checkMisbehaviourHeader uses hardcoded default trust level instead of per-client TrustLevel 23
16. Missing AppVersion field in header conversion will break client updates if Gno.land populates it 24
17. DelegateGovernor does not validate that the target governor exists 24
18. Missing else-if in UpdateGovernorStatus causes a redundant undelegate and redelegate cycle 25

19. Missing validation of TargetActiveProposals allows permanent deposit ratchet 25
20. Genesis import panics for non-governor governance delegations 26
21. Asymmetric group sizes bias the Nakamoto coefficient adjustment toward decreasing 26
22. Bankers rounding in Nakamoto bonus division can cause over-distribution and halt the chain 27
23. DeleteProposal does not remove QuorumCheckQueue entry, causing early termination of quorum checks 27
24. GetNewMinDeposit can return an unsorted sdk.Coins 28
25. Module invariants are unreachable dead code 28
26. Governor existence checks mask storage and encoding errors 29
27. InitGenesis does not validate minimum self-delegation for imported governors 29
28. MsgEditGovernor.ValidateBasic missing description length validation 30
29. Unchecked error from GetValidator can cause a nil pointer panic 30
30. V4-to-v5 migration does not clear deprecated parameters, breaking genesis export 31
31. Default minimum governor self-delegation does not match the on-chain value set by the v4 upgrade 31
32. Spelling and grammar issues across the codebase 32
33. Remove unused keeper fields on the AppModule struct 33
34. V4 upgrade handler sources the MinDepositRatio and GovernorStatusChangePeriod governance parameter defaults from the Cosmos SDK fork instead of the AtomOne definitions 33
35. Incorrect module name referenced in error comment 34
36. A vetoed proposal is not removed from the quorum check queue 34
37. No upper bound on proposal message count 35
38. Missing MsgAnnotateProposal annotation length validation 35
39. MinGovernorSelfDelegation allows zero delegations 36
40. Participation EMA values not validated in genesis 36
41. No validation that the steering DAO and oversight DAO addresses are different 37
42. Hardcoded minStakedTokens is not governance-configurable 37
43. EndBlocker ordering contradicts documentation 38
44. VerifyLightCommit error message reports incorrect voting power threshold 38
45. ValidateTrustLevel function is dead code 38
46. EditGovernor overwrites the full description instead of merging fields 39
47. Unhandled collection walk errors are silently discarded 39
48. Wrong variable in the error message for the MaximumCoefficient validation 40
49. Missing Use field for GovernanceDelegation query CLI command 40
50. Misleading --from [granter] placeholder in prune-grants CLI command 41

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by All in Bits, Inc to perform a security audit of AtomOne v4 Changes.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/atomone-hub/atomone
Label	Paths referencing this target are prefixed below with `atomone:`
Scope	The scope is restricted to the changes applied in the git diff: • https://github.com/atomone-hub/atomone/compare/a948052ffbe9b594070abe601b39f91ac1a8498c...547c4f26ffca03ed681697fd64e550ef34b30fa1 reviewed at commit `547c4f26ffca03ed681697fd64e550ef34b30fa1`, base branch at `a948052ffbe9b594070abe601b39f91ac1a8498c`.

Fixes verified at commit	`5799dca2d44d601d26ce305f10d6810a2feb2070` Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Repository	https://github.com/atomone-hub/cosmos-sdk
Label	Paths referencing this target are prefixed below with `atomone-cosmos-sdk:`
Scope	The scope is restricted to the changes applied in the git diff: • https://github.com/cosmos/cosmos-sdk/compare/5f468ffbabfeaf3e4d3c8bd733b60fda61334be4...atomone-hub:cosmos-sdk:718804a67c71726e45fc86d491bf8beb2307d246 reviewed at commit `718804a67c71726e45fc86d491bf8beb2307d246`, base branch at `5f468ffbabfeaf3e4d3c8bd733b60fda61334be4`.
Fixes verified at commit	`c61e079f2255d8693e26138fdb84b3e0820774ce` Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Repository	https://github.com/atomone-hub/atomone
Label	Paths referencing this target are prefixed below with `gno-lc:`
Scope	The scope is restricted to the changes applied in the following pull requests: • https://github.com/atomone-hub/atomone/pull/261 reviewed at commit `08f8b4ff317d0160dcc7db5f133c6923ba6e4f22`, base branch at `9ffc7482390229a84ca982d6af596689a1686756`.
Fixes verified at commit	`32fd45242560e698871581ea3a125519faff1ac7` Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

AtomOne is a Cosmos SDK-based Layer 1 blockchain forked from the Cosmos Hub.

This audit covers three parts across two repositories:

1. The core AtomOne chain: Cosmos SDK v0.50 migration, a new two-tier DAO governance module with steering and oversight DAOs, dynamic fees, etc.
2. AtomOne's Cosmos SDK fork: governor-based governance delegation, dynamic deposit throttlers, Nakamoto bonus distribution, and the epoch module
3. IBC light client module for Gno chain connectivity

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	**Low-Medium**	-
Code readability and clarity	**Medium-High**	-
Level of documentation	**Medium-High**	The client provided detailed documentation.
Test coverage	**Low**	For `atomone`, `go test -cover` reports a unit test line coverage of **5.2%** for the files in scope (excluding E2E tests, which are not reflected in this figure). For `atomone-cosmos-sdk`, `go test -cover` reports a unit test line coverage of **~11.7%** for the files in scope. For `gno-lc`, `go test -cover` reports a unit test line coverage of ~**52.9%** for the files in scope.

Summary of Findings

No	Description	Severity	Status
1	Duplicate header resubmission falsely triggers misbehavior detection and freezes the Gno light client	Critical	Resolved
2	Equal-height equivocation detection is completely disabled due to using the parent block ID instead of the commit block ID	Major	Resolved
3	`VerifyLightCommit` incorrectly validates signatures by index, breaking non-adjacent header verification	Major	Resolved
4	Proto3 `CommitSig` encoding cannot represent absent Gno.land precommits, preventing conversion of blocks	Major	Resolved
5	`MaxGovernors` parameter is defined but never checked	Major	Acknowledged
6	Inverted error check in staking hooks disables governor auto-deactivation	Major	Resolved
7	Nested authz `MsgExec` bypasses `GovVoteDecorator` minimum stake check	Major	Resolved
8	Minimum stake requirement for governance voting is bypassed via `cosmos.gov.v1` message types	Major	Resolved
9	Genesis import doubles active governor validator shares	Major	Resolved
10	Missing event emission in `UpdateParams` reduces off-chain observability	Minor	Acknowledged
11	`MsgUpdateParams` does not validate params, allowing chain-halting parameter configuration	Minor	Resolved
12	`MinLearningRate` nil check uses the wrong field in parameter validation	Minor	Resolved
13	Panic recovery in `UpdateBaseGasPrice` can leave an inconsistent state	Minor	Resolved
14	No upper bound check on minted photons relative to `MaxSupply`	Minor	Acknowledged

15	`checkMisbehaviourHeader` uses hardcoded default trust level instead of per-client `TrustLevel`	Minor	Resolved
16	Missing `AppVersion` field in header conversion will break client updates if Gno.land populates it	Minor	Resolved
17	`DelegateGovernor` does not validate that the target governor exists	Minor	Resolved
18	Missing else-if in `UpdateGovernorStatus` causes a redundant undelegate and redelegate cycle	Minor	Resolved
19	Missing validation of `TargetActiveProposals` allows permanent deposit ratchet	Minor	Resolved
20	Genesis import panics for non-governor governance delegations	Minor	Resolved
21	Asymmetric group sizes bias the Nakamoto coefficient adjustment toward decreasing	Minor	Resolved
22	Bankers rounding in Nakamoto bonus division can cause over-distribution and halt the chain	Minor	Resolved
23	`DeleteProposal` does not remove `QuorumCheckQueue` entry, causing early termination of quorum checks	Minor	Resolved
24	`GetNewMinDeposit` can return an unsorted `sdk.Coins`	Minor	Resolved
25	Module invariants are unreachable dead code	Minor	Acknowledged
26	Governor existence checks mask storage and encoding errors	Minor	Resolved
27	`InitGenesis` does not validate minimum self-delegation for imported governors	Minor	Resolved
28	`MsgEditGovernor`.ValidateBasic missing description length validation	Minor	Resolved
29	Unchecked error from `GetValidator` can cause a nil pointer panic	Minor	Resolved
30	V4-to-v5 migration does not clear deprecated parameters, breaking genesis export	Minor	Resolved
31	Default minimum governor self-delegation does not match the on-chain value set by the v4 upgrade	Informational	Acknowledged

32	Spelling and grammar issues across the codebase	Informational	Resolved
33	Remove unused keeper fields on the `AppModule` struct	Informational	Acknowledged
34	V4 upgrade handler sources the `MinDepositRatio` and `GovernorStatusChangePeriod` governance parameter defaults from the Cosmos SDK fork instead of the AtomOne definitions	Informational	Resolved
35	Incorrect module name referenced in error comment	Informational	Resolved
36	A vetoed proposal is not removed from the quorum check queue	Informational	Acknowledged
37	No upper bound on proposal message count	Informational	Acknowledged
38	Missing `MsgAnnotateProposal` annotation length validation	Informational	Resolved
39	`MinGovernorSelfDelegation` allows zero delegations	Informational	Acknowledged
40	Participation EMA values not validated in genesis	Informational	Resolved
41	No validation that the steering DAO and oversight DAO addresses are different	Informational	Resolved
42	Hardcoded `minStakedTokens` is not governance-configurable	Informational	Acknowledged
43	`EndBlocker` ordering contradicts documentation	Informational	Resolved
44	`VerifyLightCommit` error message reports incorrect voting power threshold	Informational	Resolved
45	`ValidateTrustLevel` function is dead code	Informational	Resolved
46	`EditGovernor` overwrites the full description instead of merging fields	Informational	Resolved
47	Unhandled collection walk errors are silently discarded	Informational	Resolved
48	Wrong variable in the error message for the `MaximumCoefficient` validation	Informational	Resolved
49	Missing Use field for `GovernanceDelegation` query CLI command	Informational	Resolved

50	Misleading `--from [granter]` placeholder in prune-grants CLI command	**Informational**	**Resolved**

Detailed Findings

1. Duplicate header resubmission falsely triggers misbehavior detection and freezes the Gno light client

Severity: Critical

In `gno-lc:modules/10-gno/header.go:18-24`, the `ConsensusState` function constructs a `ConsensusState` struct with only `Timestamp`, `Root`, and `NextValidatorsHash`. The `LcType` field is left unset, defaulting to an empty string.

Consensus states persisted through `UpdateState` in `gno-lc:modules/10-gno/update.go:166-170`, however, they are created via `NewConsensusState`, which sets `LcType` to `"10-gno"`. Every consensus state in the client store, therefore, carries `LcType: "10-gno"`.

In `gno-lc:modules/10-gno/misbehaviour_handle.go:30-39`, the `CheckForMisbehaviour` function retrieves the stored consensus state for a given height and compares it against the header-derived consensus state using `reflect.DeepEqual`. The stored state has `LcType: "10-gno"` while the header-derived state has `LcType: ""`, so the comparison always fails, even for a legitimate duplicate of an already-processed header. A failed comparison causes the function to conclude that two conflicting consensus states exist at the same height, returning `true` to signal misbehavior. The IBC client keeper then calls `UpdateStateOnMisbehaviour`, which freezes the client.

Any relayer or third party can exploit this to freeze an active `10-gno` IBC client by resubmitting a previously accepted header. The header passes `VerifyClientMessage` since it is cryptographically valid, but the `CheckForMisbehaviour` function incorrectly flags it due to the `LcType` mismatch. This amounts to a zero-cost denial-of-service that halts all IBC operations on the affected client until governance recovery.

Recommendation

We recommend setting `LcType: Gno` in the `ConsensusState` struct.

Status: Resolved

2. Equal-height equivocation detection is completely disabled due to using the parent block ID instead of the commit block ID

Severity: Major

In `gno-lc:modules/10-gno/misbehaviour_handle.go:59`, line `63`, as well as in `modules/10-gno/misbehaviour.go:91`, and line `95`, the code uses `Header.LastBlockId` instead of `SignedHeader.Commit.BlockId` to obtain the block

ID for misbehaviour detection. The `LastBlockId` field in a block header refers to the block ID of the parent block at height H-1, not the block that validators actually committed to at height H. The upstream [07-tendermint] correctly uses `Commit.BlockID`.

This breaks misbehaviour handling in two ways:

1. In the `CheckForMisbehaviour` function in `modules/10-gno/misbehaviour_handle.go:21-81`, block IDs are extracted from the two conflicting headers to determine whether validators signed different blocks at the same height. Because `LastBlockId` points to the shared parent block, the two values are always identical for any two blocks produced at the same height, regardless of whether validators have equivocated. The hash comparison in line `69`, therefore, never detects a fork, and the function returns `false` for all equal-height misbehaviour submissions.
2. In the `ValidateBasic` function in `modules/10-gno/misbehaviour.go:46-106`, block IDs constructed from `Header.LastBlockId` are passed to the `validCommit` function, which calls `VerifyCommit`. Inside `VerifyCommit` in `modules/10-gno/verifier.go:213`, the check `blockID.Equals(commit.BlockID)` compares the provided block ID against the commit's own `BlockID`. Since the `LastBlockId` of a block never equals the `Commit.BlockID` of that same block, this comparison always fails, and every misbehaviour submission is rejected during basic validation.

Together, these two bugs prevent any equal-height equivocation from being detected or reported. The light client can never be frozen in response to a same-height fork, so IBC relayers and connected chains have no way to react to double-signing attacks.

Recommendation

We recommend replacing all four occurrences of `Header.LastBlockId` with `SignedHeader.Commit.BlockId` to match the upstream implementation.

Status: Resolved

3. `VerifyLightCommit` incorrectly validates signatures by index, breaking non-adjacent header verification

Severity: Major

In `gno-lc:modules/10-gno/verifier.go:60`, the `VerifyNonAdjacent` function calls `VerifyLightCommit` to check that a sufficient fraction of the trusted validator set (at height X) signed the untrusted commit (at height Y). This trust-level check is what allows the light client to safely skip heights. The problem is that `VerifyLightCommit` was implemented by copying the logic from `VerifyCommit`, a function designed for same-height

verification in which the validator set and the commit are guaranteed to correspond. In the cross-height context of `VerifyNonAdjacent`, this introduces two compounding bugs.

In `gno-lc:modules/10-gno/verifier.go:207`, the size-equality check `vals.Size() != len(commit.Precommits)` requires the trusted validator set to have the same number of validators as there are precommit slots in the untrusted commit. The commit at height Y is structured according to the height-Y validator set, so any change in the validator set size between heights X and Y causes this check to reject the commit outright. Non-adjacent verification becomes impossible whenever the composition of the validator set changes.

Even if the validator set sizes coincide, `VerifyLightCommit` in line `224` uses the `vals.GetByIndex` function to look up validators by their positional index in the trusted set. The precommit array in the commit is ordered by the height-Y validator set. Since validators are sorted by address and the set composition may differ between heights, position `i` in the trusted set and position `i` in the untrusted commit generally refer to different validators. Signatures are verified against the wrong public keys, causing valid commits to be rejected or, worse, allowing an attacker to craft a commit that passes verification against unintended validators.

Recommendation

We recommend rewriting `VerifyLightCommit` to follow the same pattern as Gno tm2's `VerifyFutureCommit` function, using address-based validator lookup via `GetByAddress`, removing the size-equality check, and adding a `seen` map to prevent double-counting of voting power.

Status: Resolved

4. Proto3 `CommitSig` encoding cannot represent absent Gno.land precommits, preventing conversion of blocks

Severity: Major

In `gno-lc:modules/10-gno/helpers.go:65`, the `ConvertToGnoCommit` function iterates over the proto `Commit.Precommits` slice and attempts to skip absent validators with a `sig == nil` check in line `66`. This guard is effectively dead code. proto3 repeated message fields always deserialize to non-nil pointers in Go, so absent validators can only appear as zero-value `CommitSig` structs rather than nil entries, which leads to nil pointer panics.

The `VerifyCommit` and `VerifyLightCommit` functions in `gno-lc:modules/10-gno/verifier.go` enforce that the precommit slice length equals the validator set size. Without a way to distinguish absent from present validators in the proto encoding, a relayer cannot produce a commit that both accounts for absent validators and satisfies this size check.

Since virtually every real gno.land block has validators that did not sign, no authentic header can pass through this conversion path.

Recommendation

We recommend adding an absence value to the `CommitSig` proto message, such as a `BlockIDFlag` enum field mirroring CometBFT's amino-to-proto3 migration pattern, and updating `ConvertToGnoCommit` to detect absent entries and produce nil `Precommits` slice entries instead of accessing nested fields.

Status: Resolved

5. `MaxGovernors` parameter is defined but never checked

Severity: Major

`DefaultMaxGovernors` is defined in `atomone-cosmos-sdk:x/gov/types/v1/params.go:81` as `100`, but `CreateGovernor` in `atomone-cosmos-sdk:x/gov/keeper/msg_server.go:349` never reads or enforces this limit. Any account that meets the minimum self-delegation requirement can register as a governor without bounds.

An unbounded governor set affects tally cost and storage growth, and is misleading because governance participants may assume `MaxGovernors` is enforced. This, by itself, is classified as minor severity because the minimum delegation requirement limits the potential for abuse.

However, in combination with the ability to bypass minimum delegation checks, as reported in both "Nested authz `MsgExec` bypasses `GovVoteDecorator` minimum stake check" and "Minimum stake requirement for governance voting is bypassed via `cosmos.gov.v1` message types", this could be abused to have a large impact, thus being classified as a major severity.

Recommendation

We recommend updating `CreateGovernor` to check the current governor count against `params.MaxGovernors` before allowing registration.

Status: Acknowledged

The client acknowledges this finding, noting that this value is a leftover. It is not supposed to be checked (see also the discussion in https://github.com/atomone-hub/cosmos-sdk/pull/62).

6. Inverted error check in staking hooks disables governor auto-deactivation

Severity: Major

In `atomone-cosmos-sdk:x/gov/keeper/hooks.go:81` and `atomone-cosmos-sdk:x/gov/keeper/hooks.go:107`, both the `AfterDelegationModified` and `BeforeDelegationRemoved` functions use `err != nil` instead of `err == nil` when checking whether a delegator is an active governor. This inverts the logic: the auto-deactivation body runs only when the governor lookup fails, in which case `IsActive` returns false on the zero-value struct, and is skipped entirely when the governor exists. The deactivation logic is dead code in both hooks.

As a result, an active governor can undelegate or redelegate below the `MinGovernorSelfDelegation` threshold and remain marked `Active` indefinitely. While the `ValidateGovernorMinSelfDelegation` function in `atomone-cosmos-sdk:x/gov/keeper/tally.go:294` independently excludes under-delegated governors from vote tallying, but the stale `Active` status still causes problems. New delegators can delegate governance power to them without warning, the governor is blocked from redelegating by the active-governor check in `atomone-cosmos-sdk:x/gov/keeper/msg_server.go:534`, and the 28-day `GovernorStatusChangePeriod` cooldown never starts because `LastStatusChangeTime` is never updated.

Recommendation

We recommend changing both conditions from `err != nil` to `err == nil`.

Status: Resolved

7. Nested authz `MsgExec` bypasses `GovVoteDecorator` minimum stake check

Severity: Major

In `atomone:ante/gov_vote_ante.go`, `ValidateVoteMsgs` in `line 61` unpacks one level of authz `MsgExec` inner messages in lines `133-145` and validates each via `validMsg`. However, if an inner message is itself a `MsgExec`, `validMsg` hits the default case in `lines 87-89` and returns nil. A double-nested `MsgExec` wrapping `MsgVote` bypasses the minimum stake check entirely, allowing any account with valid authz grants to vote without meeting the `1 ATONE` minimum staking requirement.

Recommendation

We recommend either recursively validating nested `MsgExec` messages or explicitly rejecting nested `MsgExec` within the `validAuthz` function.

Status: Resolved

8. Minimum stake requirement for governance voting is bypassed via `cosmos.gov.v1` message types

Severity: Major

In `atomone:ante/gov_vote_ante.go:66`, the `GovVoteDecorator` ante decorator enforces a minimum stake requirement of 1 ATONE for voting. Its type switch matches only the four atomone-prefixed vote message types (`*govv1.MsgVote`, `*govv1.MsgVoteWeighted`, and their `v1beta1` equivalents).

However, the application also registers the AtomOne's forked Cosmos SDK governance module, which exposes `cosmos.gov.v1.Msg` and `cosmos.gov.v1beta1.Msg` service endpoints. Both modules share the same underlying keeper state. The SDK fork's vote message types, e.g., `github.com/cosmos/cosmos-sdk/x/gov/types/v1.MsgVote`, are distinct Go types that register under different protobuf type URLs: `/cosmos.gov.v1.MsgVote` versus `/atomone.gov.v1.MsgVote`.

Therefore, when a user submits a vote using the SDK's type URL, the type switch falls through to the `default` branch in line `87` and returns `nil`, skipping the minimum stake validation entirely. Any account can therefore cast governance votes without holding staked ATONE by using the `cosmos.gov.v1` message types instead of the atomone-prefixed equivalents, defeating the sybil protection this ante decorator is meant to provide.

Recommendation

We recommend adding cases for the SDK fork's vote message types to the `GovVoteDecorator` ante decorator.

Status: Resolved

9. Genesis import doubles active governor validator shares

Severity: Major

In `atomone-cosmos-sdk:x/gov/genesis.go`, the `InitGenesis` function processes the governor state in two loops. The first loop in `line 186` calls `DelegateToGovernor` for each active governor, creating a self-delegation, and then calls `IncreaseGovernorShares` for every staking delegation. The second loop in `line 213` iterates over exported `GovernanceDelegations` and calls `DelegateToGovernor` again for each entry.

Because `ExportGenesis` exports all delegations, including active governor self-delegations already handled by the first loop, the second call re-adds the full share amount. After any export-import cycle, every active governor has exactly twice their real validator shares in `ValidatorSharesByGovernor`. Since the tally in `atomone-cosmos-sdk:x/gov/keeper/tally.go:304` computes the governor's

voting power from these shares. As a result, this doubles the voting power of every active governor.

Recommendation

We recommend skipping delegations in the second loop when the delegator is an active governor already processed by the first loop

Status: Resolved

10. Missing event emission in `UpdateParams` reduces off-chain observability

Severity: Minor

In `atomone:x/coredaos/keeper/msg_server.go:31-72`, the `UpdateParams` handler is the only message handler in the coredaos module that does not emit an event upon successful execution. All other handlers (`AnnotateProposal`, `EndorseProposal`, `ExtendVotingPeriod`, `VetoProposal`) emit events, but no corresponding `EventTypeUpdateParams` constant exists.

Since `MsgUpdateParams` can modify the privileged steering DAO and oversight DAO addresses, the missing event prevents off-chain indexers and monitoring tools from detecting these changes.

Recommendation

We recommend emitting an event at the end of the `UpdateParams` handler.

Status: Acknowledged

The client acknowledges this finding, noting that it is not idiomatic in the Cosmos SDK to send events for parameter updates.

11. `MsgUpdateParams` does not validate params, allowing chain-halting parameter configuration

Severity: Minor

In `atomone:x/dynamicfee/keeper/msg_server.go:26-54`, neither `ValidateBasic` in `atomone:x/dynamicfee/types/msgs.go:25` nor the `UpdateParams` handler validates that the provided params are correct before storing them.

Governance can set invalid parameters, such as `Window=0`, which causes an out-of-bounds panic in `EndBlocker`, or `TargetBlockUtilization=0`, which causes division by zero in

`GetTargetBlockGas`. Panics in `EndBlocker` halt the chain. We classify this issue as minor due to its low likelihood, as it requires a privileged operation.

Recommendation

We recommend calling `params.ValidateBasic` in the `UpdateParams` handler before storing params. The existing validation in `params.go:41-87` already rejects invalid configurations, including zero `Window`, non-positive `TargetBlockUtilization`, and out-of-range AIMD parameters.

Status: Resolved

12. `MinLearningRate` Nil check uses the wrong field in parameter validation

Severity: Minor

In `atomone:x/dynamicfee/types/params.go:70`, the nil check uses `p.MaxLearningRate.IsNil` where it should correctly use `p.MinLearningRate.IsNil`, creating a duplicate check. As a result, `MinLearningRate` is never nil-checked, which could cause nil pointer dereferences in `InitGenesis`.

Recommendation

We recommend changing `line 70` from `p.MaxLearningRate.IsNil()` to `p.MinLearningRate.IsNil()`.

Status: Resolved

13. Panic recovery in `UpdateBaseGasPrice` can leave an inconsistent state

Severity: Minor

In `atomone:x/dynamicfee/keeper/dynamicfee.go:45-48`, `UpdateLearningRate` runs first and mutates `LearningRate`, then `UpdateBaseGasPrice` runs. If `UpdateBaseGasPrice` panics, the panic recovery resets only `BaseGasPrice`, but the already-updated `LearningRate` remains, creating a transient state inconsistency.

Recommendation

We recommend either running both updates atomically or resetting `LearningRate` in the `BaseGasPrice` panic recovery.

Status: Resolved

14. No upper bound check on minted photons relative to `MaxSupply`

Severity: Minor

In `atomone:x/photon/keeper/msg_server.go:44-54`, there is no explicit check that minted photons do not exceed `MaxSupply`. The implicit cap via the conversion rate formula is mathematically sound, but there is no explicit guard.

Recommendation

We recommend adding an explicit post-mint check that the total photon supply does not exceed `MaxSupply`.

Status: Acknowledged

The client acknowledges this finding, noting that the `MaxSupply` cannot be mathematically reached, so no change is needed. Bonded ATONE cannot be burned, and bonded ATONE tokens are needed to produce blocks. In addition, inflation constantly adds new ATONE to the supply.

15. `checkMisbehaviourHeader` uses hardcoded default trust level instead of per-client `TrustLevel`

Severity: Minor

In `gno-lc:modules/10-gno/misbehaviour_handle.go:166`, `VerifyLightCommit` uses the hardcoded `LCDefaultTrustLevel` constant (1/3) in `gno-lc:modules/10-gno/verifier.go:19` instead of the `clientState.TrustLevel` configured for the specific client instance. If a client is created with a higher trust level, such as 2/3, the misbehavior verification path still uses the weaker 1/3 threshold. A minority subset of validators that does not meet the client's configured trust threshold could still produce headers that pass misbehavior verification.

Recommendation

We recommend replacing `LCDefaultTrustLevel` on `line 166` with `clientState.TrustLevel`.

Status: Resolved

16. Missing `AppVersion` field in header conversion will break client updates if Gno.land populates it

Severity: Minor

In `gno-lc:modules/10-gno/helpers.go:105-127`, the `ConvertToGnoHeader` function converts a protobuf `GnoHeader` into a `bfttypes.Header`. While most fields are mapped over, `AppVersion` is omitted, leaving it as its zero value (empty string) in the resulting struct.

The protobuf `GnoHeader` message defines an `app_version` field (field number 7), and the generated Go struct exposes a corresponding `AppVersion` string field. On the `bfttypes` side, the `Header.Hash` function computes a Merkle hash over all header fields in order, with `AppVersion` as the seventh leaf. The hash computed from the converted header must match the one the Gno chain originally produced, which also includes `AppVersion` in its Merkle tree.

Today, this omission has no effect. Gno's `BaseApp.Info` function does not return an `AppVersion` value, so all block headers are produced with `AppVersion: ""`. Both sides feed an empty string into `bytesOrNil("")`, which returns `nil`, producing identical Merkle leaves.

The Gno application does, however, set an app version internally via `SetAppVersion("dev")`. If the `Info` function is updated to surface this value, every block header would carry a non-empty `AppVersion`, while the light client would still reconstruct the header with an empty string. The resulting Merkle hash mismatch would cause `SignedHeader.ValidateBasic` to fail, since it checks that the hash, calculated by the `Header.Hash` function, equals `Commit.BlockID.Hash`. All `UpdateClient` transactions would be rejected, bricking the light client.

Recommendation

We recommend adding `AppVersion: header.AppVersion`, to the `bfttypes.Header` struct literal in `ConvertToGnoHeader` so that the field is carried over during conversion.

Status: Resolved

17. `DelegateGovernor` does not validate that the target governor exists

Severity: Minor

The `DelegateGovernor` function in `atomone-cosmos-sdk:x/gov/keeper/msg_server.go:529` does not verify that the target governor address corresponds to an entry in the `Governors` collection before creating the governance delegation. The handler proceeds directly to call `DelegateToGovernor` in

`atomone-cosmos-sdk:x/gov/keeper/delegation.go:114`, which writes the `GovernanceDelegation` record and `GovernorValShares` entries.

If a user targets an address that was never registered via `MsgCreateGovernor`, the delegation is created successfully, and the delegator's governance voting power is silently lost. Undelegation will also cause an unexpected error, blocking the user from directly undelegating.

Recommendation

We recommend that the `DelegateGovernor` function validates the target governor exists and is `Active` before creating the delegation.

Status: Resolved

18. Missing else-if in `UpdateGovernorStatus` causes a redundant undelegate and redelegate cycle

Severity: Minor

When activating a governor with no prior governance delegation, `UpdateGovernorStatus` in `atomone-cosmos-sdk:x/gov/keeper/msg_server.go:498-514` correctly creates a self-delegation in `line 503` after detecting `ErrNotFound`. However, execution falls through to the second conditional in line `509`, which checks `delegation.GovernorAddress != govAddr.String()`. Because `delegation` is still the zero-value struct from the failed `Get`, its `GovernorAddress` is an empty string, which always differs from the governor's address. This triggers `RedelegateToGovernor`, which tears down the just-created delegation and rebuilds it. The final state is correct, but the intermediate undelegate zeroes out all governor validator share entries unnecessarily.

Recommendation

We recommend changing the condition in line `509` from `if` to `else if`.

Status: Resolved

19. Missing validation of `TargetActiveProposals` allows permanent deposit ratchet

Severity: Minor

`MinDepositThrottler.TargetActiveProposals` is greater than zero. If governance sets this parameter to `0`, the condition in `UpdateMinDeposit` in `atomone-cosmos-sdk:x/gov/keeper/min_deposit.go` becomes `numActiveProposals.GTE(targetActiveProposals)`, which is always `true`. Time-based deposit updates return early without decreasing the minimum, and per-proposal

updates always increase it. The minimum deposit can only increase and never decrease, eventually making governance proposals prohibitively expensive.

Recommendation

We recommend adding validation in `ValidateBasic`, requiring `TargetActiveProposals > 0`.

Status: Resolved

20. Genesis import panics for non-governor governance delegations

Severity: Minor

In `atomone-cosmos-sdk:x/gov/genesis.go:209`, the check has an inverted error condition. The intent is to prevent active governors from delegating to other governors.

However, `err != nil` is `true` when the delegator is not in the governor set, and since a regular delegator's address differs from the target governor, `!delGovAddr.Equals(govAddr)` is also `true`. The function panics with "account X is an active governor and cannot delegate" for any non-governor delegator. Since `ExportGenesis` exports all governance delegations, including those from non-governor accounts, any chain with such delegations will halt on genesis import.

Additionally, it lacks a check to verify if the governor is active by calling the `IsActive` function.

Recommendation

We recommend changing `err != nil` to `err == nil` and checking if the governor is active.

Status: Resolved

21. Asymmetric group sizes bias the Nakamoto coefficient adjustment toward decreasing

Severity: Minor

In `atomone-cosmos-sdk:x/distribution/keeper/nakamoto_bonus.go:55-57`, the `AdjustNakamotoBonusCoefficient` function splits validators into three groups using `groupSize = n / 3`. The `high` group takes the first `groupSize` validators, while the `low` group takes all validators from index `groupSize*2` onward, absorbing any remainder when `n` is not divisible by 3.

Since validators are sorted by bonded tokens in descending order, the extra validators in the `low` group are mid-tier ones that raise their average. The adjustment logic in line `78` compares `highAvg / lowAvg`, so a higher `low` average makes it harder to reach the 3x threshold for increasing the coefficient, systematically biasing the mechanism toward decreasing it.

Recommendation

We recommend using symmetric group sizes by slicing `low` from the end of the list (`low := validators[n-groupSize:]`) so both groups always have the same size.

Status: Resolved

22. Bankers rounding in Nakamoto bonus division can cause over-distribution and halt the chain

Severity: Minor

In `atomone-cosmos-sdk:x/distribution/keeper/allocation.go:74` and line `90`, per-validator amounts are computed using `Quo`, which internally applies bankers' rounding and can round up. When the rounded-up amount is distributed to every validator in the loop in lines `83-101`, the cumulative total could exceed available fees. Subsequently, in line `100`, `DecCoins.Sub` panics with `"negative coin amount"` if the result would be negative. Since `AllocateTokens` runs during `BeginBlock`, this panic would halt the chain.

Recommendation

We recommend replacing `Quo` with `QuoTruncate` in lines `74` and `90`.

Status: Resolved

23. `DeleteProposal` does not remove `QuorumCheckQueue` entry, causing early termination of quorum checks

Severity: Minor

In `atomone-cosmos-sdk:x/gov/keeper/proposal.go:276`, the `ActivateVotingPeriod` function adds entries to the `QuorumCheckQueue`. However, the `DeleteProposal` function in line `215` does not remove the corresponding entry. While it properly cleans up `InactiveProposalsQueue`, `ActiveProposalsQueue`, and `VotingPeriodProposals`, `QuorumCheckQueue` is missed.

When a deleted proposal's stale entry is processed by `EndBlocker` in `atomone-cosmos-sdk:x/gov/abci.go:108-198`, the `Walk` callback fails on

`Proposals.Get` for the missing proposal and returns early, skipping all remaining proposals due for a quorum check at the same block, which will then be processed in the next block.

Recommendation

We recommend removing the `QuorumCheckQueue` entry in the `DeleteProposal` function.

Status: Resolved

24. `GetNewMinDeposit` can return an unsorted `sdk.Coins`

Severity: Minor

In `atomone-cosmos-sdk:x/gov/types/v1/min_deposit.go:9-43`, the `GetNewMinDeposit` function constructs a `sdk.Coins` slice by first appending existing denominations, then new floor denominations. If a new floor denom sorts alphabetically before an existing one, the result is unsorted.

Recommendation

We recommend constructing the result using `sdk.NewCoins(newMinDeposit...)`, which normalizes and sorts the coins.

Status: Resolved

25. Module invariants are unreachable dead code

Severity: Minor

In `atomone-cosmos-sdk:x/gov/keeper/invariants.go:16-18`, the `RegisterInvariants` function registers `ModuleAccountInvariant` and `GovernorsDelegationsInvariant`.

However, these invariants can never execute. The `x/crisis` module, which provides the `InvariantRegistry`, has been reduced to a bare placeholder with no functional code. These invariants were reintroduced alongside the governors feature after the upstream SDK deliberately removed all module invariants, but without the supporting infrastructure, they provide no safety guarantees.

Recommendation

We recommend removing the unreachable invariant code to avoid confusion and maintain code clarity.

If invariants are desired in the future, they should be re-implemented with the necessary supporting infrastructure to ensure they can be executed and provide value.

Status: Acknowledged

The client acknowledges this finding, noting that the `x/crisis` module was removed and, in fact, **no invariants** in the entire codebase are ever checked. The invariants are added for consistency, but they are meant to be unused code, possibly used in the future should a revised crisis module be added again. See also https://github.com/atomone-hub/atomone/pull/93 and the included links in the PR description that justify the removal.

26. Governor existence checks mask storage and encoding errors

Severity: Minor

In `atomone-cosmos-sdk:x/gov/keeper/msg_server.go:353`, the `CreateGovernor` function treats any non-nil error from `k.Governors.Get` as if the governor could not be found. The same pattern appears in the `EditGovernor` function in line `422` and `UpdateGovernorStatus` in line `454`.

However, the `collections.Map.Get` function can also return encoding or lower-level storage errors.

Recommendation

We recommend explicitly checking for `collections.ErrNotFound` using `errors.Is`.

Status: Resolved

27. `InitGenesis` does not validate minimum self-delegation for imported governors

Severity: Minor

In `atomone-cosmos-sdk:x/gov/genesis.go:176-191`, the `InitGenesis` function stores governors from the genesis state and creates self-delegations for active ones via the `DelegateToGovernor` function. However, it does not verify that an active governor's bonded tokens meet the `MinGovernorSelfDelegation` parameter.

This is inconsistent with the `MsgCreateGovernor` handler in `atomone-cosmos-sdk:x/gov/keeper/msg_server.go:370-382` and the

`UpdateGovernorStatus` function in lines `485-488`, both of which validate `MinGovernorSelfDelegation` before allowing governor creation or activation.

Recommendation

We recommend adding a `ValidateGovernorMinSelfDelegation` check for active governors in the `InitGenesis` loop.

Status: Resolved

28. `MsgEditGovernor.ValidateBasic` missing description length validation

Severity: Minor

In `atomone-cosmos-sdk:x/gov/types/v1/msgs.go:156-162`, the `ValidateBasic` function for the `MsgEditGovernor` message only validates the governor address but does not call `EnsureLength` on the description.

While the server-side handler in `atomone-cosmos-sdk:x/gov/keeper/msg_server.go:427` enforces the length constraint before persisting, the missing `ValidateBasic` check allows transactions with arbitrarily long description fields to enter the mempool and propagate through the network before being rejected at execution time.

Recommendation

We recommend calling `EnsureLength` in the `ValidateBasic` function.

Status: Resolved

29. Unchecked error from `GetValidator` can cause a nil pointer panic

Severity: Minor

In `atomone-cosmos-sdk:x/gov/keeper/governor.go:24`, the `getGovernorBondedTokens` function calls `keeper.sk.GetValidator` to retrieve a validator during delegation iteration, but the returned error is silently discarded.

When `GetValidator` does not find a validator for a given address, it returns a zero-value `Validator` struct along with an `ErrNoValidatorFound` error. In line `26`, the subsequent `validator.GetBondedTokens` and `validator.GetDelegatorShares` functions are called on this zero-value struct, and the `Quo` call attempts to divide by the uninitialized `DelegatorShares`, which triggers a nil pointer dereference panic.

While a delegation without a corresponding validator in the store would represent an inconsistent state, the current code does not handle it gracefully.

Recommendation

We recommend checking the error returned by `GetValidator` and propagating the error if the validator is not found.

Status: Resolved

30. V4-to-v5 migration does not clear deprecated parameters, breaking genesis export

Severity: Minor

In `atomone-cosmos-sdk:x/gov/migrations/v5/store.go:62`, the `MigrateStore` function populates new v5 governance parameters but does not zero out deprecated v4 fields:

- `MinDeposit`
- `Quorum`
- `ConstitutionAmendmentQuorum`
- `LawQuorum`
- `MinInitialDepositRatio`

The v5 `ValidateBasic` function in `atomone-cosmos-sdk:x/gov/types/v1/params.go:235-246` explicitly rejects non-empty values for these deprecated fields. As a result, any genesis export after migration produces a file that fails `validate-genesis`, breaking the standard chain upgrade workflow in which operators export state at a halt height and restart with the new binary.

Recommendation

We recommend clearing the deprecated fields in `MigrateStore` before writing the updated params back to the store.

Status: Resolved

31. Default minimum governor self-delegation does not match the on-chain value set by the v4 upgrade

Severity: Informational

In `atomone:x/gov/types/v1/params.go:76`, the `DefaultMinGovernorSelfDelegation` constant is set to `math.NewInt(1000_000000)`, which corresponds to 1,000 ATONE. However, the v4

upgrade handler in `atomone:app/upgrades/v4/upgrades.go:52` explicitly sets `MinGovernorSelfDelegation` to `math.NewInt(10000_000000)`, i.e., 10,000 ATONE.

Recommendation

We recommend updating `DefaultMinGovernorSelfDelegation` to `math.NewInt(10000_000000)` so that the default value is consistent with the on-chain parameter set by the v4 upgrade handler.

Status: Acknowledged

The client acknowledges this finding, noting that this is intentional. The default param value is not meant to match the on-chain set param. It is meant to be a representative value that requires tweaking for the specific case.

32. Spelling and grammar issues across the codebase

Severity: Informational

Multiple spelling and grammar errors exist in code comments, error messages, and an exported constant name:

- `atomone:x/coredaos/keeper/msg_server.go:382`: "veoted" should be "vetoed"
- `atomone:x/coredaos/keeper/msg_server.go:51,65`: "DAO have bonded" should be "DAO has bonded"
- `atomone:x/gov/types/v1/params.go:57`: exported constant `DefaultBurnVoteQuorom` should be `DefaultBurnVoteQuorum` (also referenced in line 186)
- `atomone:x/gov/types/v1/params.go:327, 330, 333, 341, 344, 347`: "mininum" should be "minimum"
- `atomone:x/gov/types/v1/params_legacy.go:56, 59`: "quorom" should be "quorum"
- `atomone:x/gov/types/v1/governor.go:103`: "vovernor's" should be "governor's"
- `atomone:app/upgrades/v4/upgrades.go:52`: "elegible" should be "eligible"

Recommendation

We recommend fixing all identified spelling and grammar issues across all codebases.

Status: Resolved

33. Remove unused keeper fields on the `AppModule` struct

Severity: Informational

In `atomone:x/coredaos/module.go:104-112`, the `AppModule` struct declares `accountKeeper` and `bankKeeper` fields. These two keepers are injected through `NewAppModule` in line `114` and via the dependency injection path in `ProvideModule` in line `214`, but they are never referenced by any production logic in the module. The message server only uses `govKeeper` and `stakingKeeper`.

Similarly, the genesis and hook handlers do not reference `accountKeeper` or `bankKeeper`.

Recommendation

We recommend removing `accountKeeper` and `bankKeeper` from the `AppModule` struct.

Status: Acknowledged

The client acknowledges this finding, noting that the suggestion regards the removal of `accountKeeper` and `bankKeeper` from the coredao's `AppModule`. The claim is that they are not used in any logic in the module. However, these fields are used in the simulation tests.

34. V4 upgrade handler sources the `MinDepositRatio` and `GovernorStatusChangePeriod` governance parameter defaults from the Cosmos SDK fork instead of the AtomOne definitions

Severity: Informational

In `atomone:app/upgrades/v4/upgrades.go:47`, the v4 upgrade handler initializes several AtomOne-specific governance parameters from `sdkgovv1.DefaultParams`, which resolves to the Cosmos SDK fork's default parameter definitions. AtomOne maintains its own canonical parameter defaults in `atomone:x/gov/types/v1/params.go` through the `DefaultParams` function, which defines values for `MinDepositRatio`, and `GovernorStatusChangePeriod`, among others.

The default parameter values may differ between the SDK fork and AtomOne's own definitions. Therefore, it is advisable for the upgrade handler to source these values from AtomOne's own `DefaultParams` to ensure consistency and avoid unintended discrepancies.

Recommendation

We recommend updating the `initGovParams` function to derive the two AtomOne-specific fields, `MinDepositRatio` and `GovernorStatusChangePeriod`, from AtomOne's own `v1.DefaultParams` in `atomone:x/gov/types/v1/params.go`.

Status: Resolved

35. Incorrect module name referenced in error comment

Severity: Informational

In `atomone:x/coredaos/types/errors.go:7`, the comment reads `// x/photon module sentinel errors`. This is a copy-paste artifact from `atomone:x/photon/types/errors.go`, where the identical comment correctly references the `x/photon` module. In the `x/coredaos` context, the comment is inaccurate and could mislead developers or auditors into believing the errors defined in this file belong to the `x/photon` module rather than x/coredaos.

Recommendation

We recommend updating the comment in `atomone:x/coredaos/types/errors.go:7` to `// x/coredaos module sentinel errors` to accurately reflect the module it belongs to.

Status: Resolved

36. A vetoed proposal is not removed from the quorum check queue

Severity: Informational

In `atomone:x/coredaos/keeper/msg_server.go:392`, the `VetoProposal` handler removes the proposal from the `ActiveProposalQueue` but does not remove it from the `QuorumCheckQueue`. This leaves a stale entry that the governance module's `EndBlocker` will process on a subsequent block.

In practice, this omission does not cause state corruption. When the stale `QuorumCheckQueue` entry fires, the `EndBlocker` finds that `HasReachedQuorum` returns `false` (because `DeleteVotes` already deleted all votes in line `391`) and that `VotingEndTime` is already in the past (set to the block time of the veto in line `388`). The entry is therefore silently cleaned up without modifying the proposal state.

Recommendation

We recommend removing the vetoed proposal from the `QuorumCheckQueue` at the time of veto.

Status: Acknowledged

The client acknowledges this finding, noting that, unfortunately, removing the proposal from the quorum check queue is not straightforward because the key has a dynamically computed time component that cannot be easily determined at veto time without relatively complex logic (or by traversing the entire quorum check queue). Because the proposal is still silently removed from the queue when the check is scheduled (which happens once and should have minimal impact), we think that, although not ideal, what we already have is sufficient, avoiding unnecessarily complex removal logic at veto time.

37. No upper bound on proposal message count

Severity: Informational

In `atomone:x/gov/types/v1/msgs.go`, `MsgSubmitProposal.ValidateBasic` checks that `Messages` is non-empty but enforces no maximum count. A proposal with an excessive number of embedded messages would be accepted, stored on-chain, and executed sequentially in `EndBlocker`. A sufficiently large message list could cause execution to exceed the block gas limit, making a legitimately passed proposal fail at execution time with no way to retry.

Recommendation

We recommend adding a reasonable upper bound on message count in `ValidateBasic`.

Status: Acknowledged

The client acknowledges this finding, noting that this issue arises only if a problematic proposal passes.

38. Missing `MsgAnnotateProposal` annotation length validation

Severity: Informational

In `atomone:x/coredaos/types/msgs.go:32-39`, `MsgAnnotateProposal.ValidateBasic` checks that the annotation is non-empty but enforces no maximum length. The annotation is stored on-chain, which may cause state bloat affecting every node's storage and proposal query performance.

Recommendation

We recommend adding a `MaxAnnotationLength` bound in `ValidateBasic`.

Status: Resolved

39. `MinGovernorSelfDelegation` allows zero delegations

Severity: Informational

The `MinGovernorSelfDelegation` parameter does not prevent zero-value configurations. When set to `0` via governance, any account can register as a governor without staking any tokens, since the strict less-than validation used in `CreateGovernor` in `atomone:x/gov/keeper/msg_server.go:375`, `UpdateGovernorStatus` in `atomone:x/gov/keeper/msg_server.go:486`, and `ValidateGovernorMinSelfDelegation` in `atomone:x/gov/types/v1/governor.go:63` passes when both the bonded balance and the minimum are zero.

Since governors inherit the governance voting power of all accounts that delegate to them, a zero-stakes governor can accumulate significant voting influence with no economic risk, undermining the stake-based security model.

Recommendation

We recommend adding a validation that rejects `MinGovernorSelfDelegation` values of zero.

Status: Acknowledged

The client acknowledges this finding, noting that this is intentional. It would mean the feature is effectively disabled, allowing anyone with any amount of bonded tokens (even 0) to become a governor.

40. Participation EMA values not validated in genesis

Severity: Informational

In `atomone:x/gov/types/v1/genesis.go:47-152`, `ValidateGenesis` validates proposals, deposits, votes, governors, delegations, and params, but never validates the three participation EMA fields: `ParticipationEma`, `ConstitutionAmendmentParticipationEma`, and `LawParticipationEma`.

These values feed directly into the dynamic quorum formula in `atomone:x/gov/keeper/participation_ema.go:111`. A value greater than `1.0` pushes `quorum` above `max_quorum`, potentially above `100%`, making proposals impossible

to pass. A negative value pushes quorum below `min_quorum`, making proposals trivially easy to pass. A non-decimal string causes a panic in `InitGenesis` time.

Recommendation

We recommend adding validation in `ValidateGenesis` that parses each EMA field and rejects values outside the range `[0, 1]`.

Status: Resolved

41. No validation that the steering DAO and oversight DAO addresses are different

Severity: Informational

In `atomone:x/coredaos/types/params.go:44` and the `UpdateParams` handler in `atomone:x/coredaos/keeper/msg_server.go:31`, there is no check that `SteeringDaoAddress` is not equal to `OversightDaoAddress`. Setting both to the same address would give a single entity both endorsement power to lower law proposal thresholds and veto power to reject any proposal, undermining the two-tier governance model.

Recommendation

We recommend adding a check in `ValidateBasic` that `SteeringDaoAddress` is not equal to `OversightDaoAddress` when both are non-empty.

Status: Resolved

42. Hardcoded `minStakedTokens` is not governance-configurable

Severity: Informational

In `atomone:ante/gov_vote_ante.go:21-22`, `minStakedTokens` is set to `1000000` as a hardcoded package variable. It cannot be adjusted via governance parameter changes, only through a code upgrade.

Recommendation

We recommend making this a configurable governance parameter.

Status: Acknowledged

The client acknowledges this finding, noting that they prefer the hard-coded variable since the min-stake requirement represents an anti-airdrop-chasing mechanism (since in the past airdrops were issued to voting accounts and a lot of dust accounts started voting on other

cosmos chains for that reason) and the 1 ATONE requirement is inherited from the 1 ATOM requirement present also on the cosmos hub. A governance-defined parameter would require additional logic and necessity to read from the state, which, for the purpose of the feature, seems unnecessary

43. `EndBlocker` ordering contradicts documentation

Severity: Informational

In `atomone:app/modules.go`, the comment in lines `164-171` states `gov.EndBlock` must be executed before `staking.EndBlock`, but `orderEndBlockers` in line `175` places `stakingtypes.ModuleName` before `govtypes.ModuleName` in line `176`. This contradicts the documented requirement for Interchain Security.

Recommendation

We recommend either fixing the ordering to match the comment or updating the comment to reflect the intended behavior.

Status: Resolved

44. `VerifyLightCommit` error message reports incorrect voting power threshold

Severity: Informational

In `gno-lc:modules/10-gno/verifier.go:249`, the error message when insufficient voting power is detected hardcodes the needed threshold as `vals.TotalVotingPower()*2/3+1`. The actual threshold used in the comparison in line `246` is computed from the `trustLevel` parameter. When the trust level is not 2/3, the error message reports an incorrect required threshold.

Recommendation

We recommend replacing the hardcoded expression in the error message with the computed `votingPowerNeeded` variable.

Status: Resolved

45. `ValidateTrustLevel` function is dead code

Severity: Informational

In `gno-lc:modules/10-gno/verifier.go:194-201`, a local `ValidateTrustLevel` function is defined but never called in production code.

`gno-lc:modules/10-gno/client_state.go:125` uses `light.ValidateTrustLevel` from the cometbft library instead. The local function is only referenced in tests.

Recommendation

We recommend removing the dead `ValidateTrustLevel` function or replacing the cometbft library call with the local implementation.

Status: Resolved

46. `EditGovernor` overwrites the full description instead of merging fields

Severity: Informational

The `EditGovernor` function in `atomone-cosmos-sdk:x/gov/keeper/msg_server.go:417` replaces the entire governor description with the message's contents rather than merging individual fields.

A dedicated `UpdateDescription` function is defined in `atomone-cosmos-sdk:x/gov/types/v1/governor.go`, which uses the `DoNotModify` parameter, allowing callers to update a single field while preserving the rest. However, this function is not used. A governor who submits a `MsgEditGovernor` message to change only one field will silently blank out every other field.

Recommendation

We recommend that `EditGovernor` calls `UpdateDescription` to merge fields in a manner consistent with the staking module.

Status: Resolved

47. Unhandled collection walk errors are silently discarded

Severity: Informational

The phase 2 quorum check `walk` in `atomone-cosmos-sdk:x/gov/abci.go:108` does not capture its return error, unlike phase 1 in `line 32` and phase 3 in `line 202`, which both check for errors. If any operation inside the callback fails, the error is silently swallowed, and `EndBlocker` returns success. This can leave proposals in inconsistent states.

Similarly, in `atomone-cosmos-sdk:x/gov/keeper/msg_server.go:622`, the error from calling `GovernanceDelegationsByGovernor.Walk` is silently discarded. If `Walk` fails, delegations remains empty, causing `len(delegations) == 0` to evaluate to `true`

and the governor to be incorrectly removed despite still having active delegations. The error should be propagated.

Recommendation

We recommend assigning the `walk`'s return value to `err` and returning it.

Status: Resolved

48. Wrong variable in the error message for the `MaximumCoefficient` validation

Severity: Informational

In `atomone-cosmos-sdk:x/distribution/types/params.go:96`, the `validateNakamotoBonus` function uses the `v.Step` variable instead of `v.MaximumCoefficient` in the error message for invalid `MaximumCoefficient` values. This seems to be a copy-paste error from the `Step` validation in line `78`.

Recommendation

We recommend replacing `v.Step` with `v.MaximumCoefficient` in line `96`.

Status: Resolved

49. Missing `Use` field for `GovernanceDelegation` query CLI command

Severity: Informational

In `atomone-cosmos-sdk:x/gov/autocli.go:112-119`, the `GovernanceDelegation` query CLI command definition omits the `Use` field. Without it, autocli derives the subcommand name as `governance-delegation`, which conflicts with the `Example` field, which shows `query gov delegation`. A user following the example would encounter an "unknown command" error.

Recommendation

We recommend setting `Use: "delegation [delegator_address]"` to match the `Example` field and the sibling `delegations` command in line `104`.

Status: Resolved

50. Misleading `--from [granter]` placeholder in `prune-grants` CLI command

Severity: Informational

In `atomone-cosmos-sdk:x/authz/module/autocli.go:75`, the `Use` field for the `PruneExpiredGrants` CLI command reads `prune-grants --from [granter]`. However, the underlying `MsgPruneExpiredGrants` proto defines its signer as `Pruner`, not `Granter`. Any account can prune expired grants. The `[granter]` placeholder misleads users into thinking the command is restricted to granters.

Recommendation

We recommend changing the placeholder to `--from [pruner]`.

Status: Resolved